



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS DE CRATEÚS

Professor: Rafael Martins Barros

Equipe: João Ismael Gonçalves Ferreira 557277

Igor Santana Sampaio 540143

Trabalho

Algoritmos em grafos

Introdução

O problema da Árvore Geradora Mínima (Minimum Spanning Tree - MST) é um dos problemas mais emblemáticos e exhaustivamente estudados na Teoria dos Grafos e na Otimização Combinatória. Considere um grafo conexo, não direcionado e ponderado $G=(V,E)$, onde V representa o conjunto de vértices, E representa o conjunto de arestas e cada aresta possui um peso positivo associado. O objetivo consiste em determinar um subconjunto de arestas que conecte todos os vértices de V , não contenha ciclos e possua a mínima soma total de pesos possível.

Este relatório técnico apresenta o desenvolvimento, a implementação computacional e a análise comparativa de desempenho de dois algoritmos clássicos para a resolução deste problema: o Algoritmo de Kruskal e o Algoritmo de Prim. Através de experimentos controlados sobre instâncias com diferentes densidades e distribuições de pesos, busca-se interpretar o comportamento empírico de ambas as abordagens à luz da complexidade teórica discutida em aula.

Fundamentação Teórica

A garantia de otimização tanto do algoritmo de Kruskal quanto do algoritmo de Prim repousa fundamentalmente na "Propriedade do Corte" das árvores geradoras. Contudo, as diferenças estruturais na forma como cada algoritmo explora o grafo geram impactos distintos de processamento:

- Algoritmo de Kruskal: Adota uma estratégia construtiva baseada em arestas. O método explora o fato de que o algoritmo de Kruskal depende fortemente da ordenação das arestas, avaliando-as de forma gulosa do menor para o maior peso. Para garantir que a adição de uma aresta não feche um ciclo na Árvore Geradora Mínima (AGM), o algoritmo utiliza a estrutura de dados auxiliar de conjuntos disjuntos (Union-Find).
- Algoritmo de Prim: Adota uma estratégia baseada em vértices. Partindo de um nó inicial arbitrário, o algoritmo faz crescer uma única árvore anexando, a cada passo, a aresta de menor peso que conecta a fronteira da árvore a um vértice ainda não visitado. Consequentemente, o desempenho do algoritmo de Prim depende da eficiência da estrutura de prioridade utilizada para gerenciar as arestas candidatas.

- **Impacto da Topologia:** A densidade do grafo definida pela razão entre o número de arestas e o número de vértices influencia diretamente o desempenho relativo dos algoritmos. Enquanto cenários esparsos favorecem abordagens focadas em ordenação global, cenários altamente densos tendem a expor gargalos estruturais distintos em cada método.

Metodologia e Implementação

Em conformidade com as restrições da disciplina, que vedam o uso de bibliotecas prontas de modelagem de grafos ou de otimização, o sistema foi codificado em Python utilizando estritamente coleções e estruturas nativas da linguagem.

Estruturas de Dados Utilizadas

- **Tuplas (tuple):** Utilizadas para representar imutavelmente as arestas ponderadas na notação (u, v, peso) e as coordenadas cartesianas dos vértices. A imutabilidade garante integridade referencial e acesso indexado em tempo constante $O(1)$
- **Listas (list):** Empregadas para armazenar as coleções de arestas do grafo, o conjunto de arestas pertencentes à AGM e as métricas de tempo durante as execuções independentes.
- **Dicionários (dict) e Conjuntos (set):** Utilizados para o mapeamento de fatias de grafos, controle de nós visitados e representação interna de florestas disjuntas.

Modelagem e Geração de Instâncias (graph_generator.py)

Para avaliar o comportamento dos algoritmos, construiu-se um gerador modular capaz de produzir três padrões de grafos:

1. **Grafos Esparsos:** Grafos com número reduzido de arestas, construídos a partir de uma árvore geradora inicial estritamente linear (para garantir conectividade) acoplada a arestas aleatórias extras até atingir a proporção de $|E| \approx |V|$.
2. **Grafos Densos:** Grafos com grande quantidade de arestas, gerados através da ligação combinatória de todos os pares possíveis de vértices, tal que $|E| \approx |V|^2$. Os pesos são gerados aleatoriamente em um intervalo definido entre 1 e 100.
3. **Grafos Geométricos:** Grafos derivados de pontos dispersos aleatoriamente no plano cartesiano bidimensional. Os pesos das arestas representam estritamente as distâncias euclidianas entre os

pares de coordenadas. Para evitar imprecisões de coerção nativa de ponto flutuante, os cálculos aplicam a fórmula de truncamento e precisão geométrica:

$$d_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Detalhamento das Funções Implementadas

kruskal.py (Implementação de Igor Santana)

- Classe **UnionFind**: Gerencia a estrutura de conjuntos disjuntos. A operação de busca (find) implementa a técnica de *compressão de caminho*, reatribuindo os nós intermediários diretamente à raiz da árvore. A operação de mesclagem (union) aplica a técnica de *união por rank*, balanceando a altura das árvores ao pendurar a raiz de menor profundidade sob a raiz mais profunda.
- Função **kruskal(num_vertices, edges)**: Recebe a lista de arestas brutas e aplica a ordenação nativa `sorted()` baseada na chave de pesos. Em seguida, itera sobre as arestas ordenadas invocando o método `union()`. Se a inserção for bem-sucedida (indicando ausência de ciclos), a aresta é agregada à solução. O laço possui uma otimização de parada antecipada assim que a árvore atinge exatamente $|V|-1$ arestas conectadas.

prim.py (Implementação de João Ismael)

- Função **prim(num_vertices, arestas)** inicializa uma lista de adjacências para mapear eficientemente as conexões do grafo, realizando a conversão da lista linear de arestas recebida. A estratégia de busca gulosa utiliza uma fila de prioridades baseada no módulo nativo `heapq` (Min-Heap) do Python, inicializada com o vértice inicial genérico 0, peso de custo 0 e sem nó predecessor (marcado com -1). Um vetor booleano de controle (visitados) rastreia quais nós já foram incorporados à estrutura da árvore. A cada iteração, o vértice que apresenta a aresta de menor peso conectada à fronteira atual da árvore é extraído do Min-Heap por meio da função `heapq.heappop()`. Caso o nó extraído ainda não tenha sido visitado, ele é inserido na Árvore Geradora Mínima (AGM), seu custo é acumulado no peso total e todos os seus vizinhos não visitados são inseridos na fila de prioridades com seus respectivos pesos. O laço principal conta com uma otimização de parada antecipada que interrompe o processamento assim que a solução atinge exatamente $|V| - 1$ arestas conectadas.

Complexidade Computacional

Algoritmo de Kruskal

- **Complexidade Temporal:** A ordenação prévia das arestas domina o custo computacional do algoritmo, exigindo tempo $O(E \log E)$. O laço principal executa E iterações, efetuando operações de busca e mesclagem na estrutura *Union-Find*. Graças às otimizações de compressão de caminho e união por rank, as operações disjuntas rodam em tempo amortizado quase constante $O(\alpha(V))$, onde α é o inverso da função de Ackermann. Sabendo que em qualquer grafo simples $\log E \leq \log V^2 = 2 \log V$, a complexidade temporal assintótica do Kruskal consolida-se em $O(E \log V)$.
- **Complexidade Espacial:** O consumo de memória é estritamente linear, classificado em $O(V + E)$ para o armazenamento da lista de arestas e dos vetores internos de parentesco e rank.

Algoritmo de Prim

- **Complexidade Temporal:** A inicialização das estruturas consome tempo linear $O(V + E)$. O laço principal realiza V extrações do Min-Heap com custo $O(V \log V)$, enquanto o relaxamento e inserção das arestas na fila exige até E operações, custando $O(E \log V)$. Assim, o custo computacional dominante consolida-se em $O(E \log V)$. Em cenários altamente densos ($|E| \approx |V|^2$), esse comportamento escala assintoticamente para o limite superior de $O(V^2 \log V)$.
- **Complexidade Espacial:** Classifica-se como linear $O(V + E)$. Esse consumo de memória é determinado pelo armazenamento estrutural da lista de adjacências e pelo tamanho máximo que a fila de prioridades pode atingir durante a execução.

Método Experimental e Resultados

Metodologia de Medição

A avaliação experimental foi conduzida através do script de automação `main.py` acoplado ao módulo `benchmark.py`. Para isolar o tempo de processamento real do algoritmo de ruídos de hardware ou de gargalos de I/O, os tempos foram medidos separadamente para a leitura/construção do grafo e para a execução do algoritmo. A captura temporal utilizou a função `time.perf_counter()`, que acessa os registradores de clock de alta precisão do sistema operacional. Cada configuração de grafo foi gerada e executada ao

menos 10 vezes consecutivas, reportando-se a média aritmética e o desvio padrão.

Os testes de escala seguiram a sugestão da especificação, avaliando instâncias crescentes de tamanho $|V| = 10^2, 5 \times 10^2, 10^3, 5 \times 10^3, 10^4$. Por limitações físicas de memória RAM e de tempo hábil de execução uma vez que em grafos densos o número de arestas cresce quadraticamente, as instâncias densas e geométricas foram avaliadas até o limite seguro de 10^3 vértices.

Discussão dos Resultados Obtidos

tipo	vertices	arestas	tempo_geracao_medio	tempo_geracao_desvio_padrao	tempo_execucao_medio	tempo_execucao_desvio_padrao
esparso	100	199	1.323.600.037.721.910	1,17E+11	6,77E+10	9,00E+09
esparso	500	999	7.225.599.983.939.900	5,19E+10	4.124.899.976.886.800	6,99E+10
esparso	1000	1999	14.234.499.976.737.400	10.266.846.956.527.600	8.774.000.045.377.760	1.050.123.999.511.350
esparso	5000	9999	830.406.999.157.276	339.775.576.360.148	4.832.610.004.814.340	27.493.533.675.479.400
esparso	10000	19999	1.800.486.000.138.330	12.793.444.359.071.300	10.336.750.009.446.400	2.165.281.545.850.360
denso	100	4950	13.790.199.969.662.300	13.587.652.956.943.100	604.489.998.659.119	6,50E+10
denso	500	124750	363.618.800.038.239	7.018.358.404.778.850	1.753.065.000.521.010	773.003.251.287.524
denso	1000	499500	15.765.658.000.309.400	7.636.604.950.590.170	7.399.506.999.645.380	4.736.598.934.602.630
geometrico	100	4950	11.828.700.022.306.200	12.590.899.791.296.400	9.693.300.002.254.540	3.909.535.352.973.510
geometrico	500	124750	3.005.331.999.738.700	1.122.983.958.489.250	25.419.079.992.570.900	13.950.651.831.065.900
geometrico	1000	499500	13.752.133.999.951.100	5.269.009.483.316.660	11.437.706.999.713.500	4.262.394.394.202.570

Tabela de Resultados resultados_kruskal.csv

Tipo	Vértices (V)	Arestas (E)	Tempo Geração Médio (s)	Tempo Geração DP (s)	Tempo Execução Médio (s)	Tempo Execução DP (s)
Esparso	100	199	0,000128	0,000021	0,000104	0,000011
Esparso	500	999	0,000695	0,000155	0,000635	0,000040
Esparso	1.000	1.999	0,001357	0,000178	0,001559	0,000349
Esparso	5.000	9.999	0,007812	0,000415	0,008876	0,000709
Esparso	10.000	19.999	0,016746	0,001475	0,027774	0,013046
Denso	100	4.950	0,001182	0,000158	0,001832	0,000425
Denso	500	124.750	0,033654	0,001374	0,086105	0,002859
Denso	1.000	499.500	0,152646	0,002056	0,368636	0,011787
Geometrico	100	4.950	0,001699	0,000896	0,002306	0,001017
Geometrico	500	124.750	0,038521	0,002509	0,113355	0,002817
Geometrico	1.000	499.500	0,168973	0,002684	0,509245	0,022161

Tabela de Resultados resultados_prim.csv

Gráfico de desempenho ESPARSOS

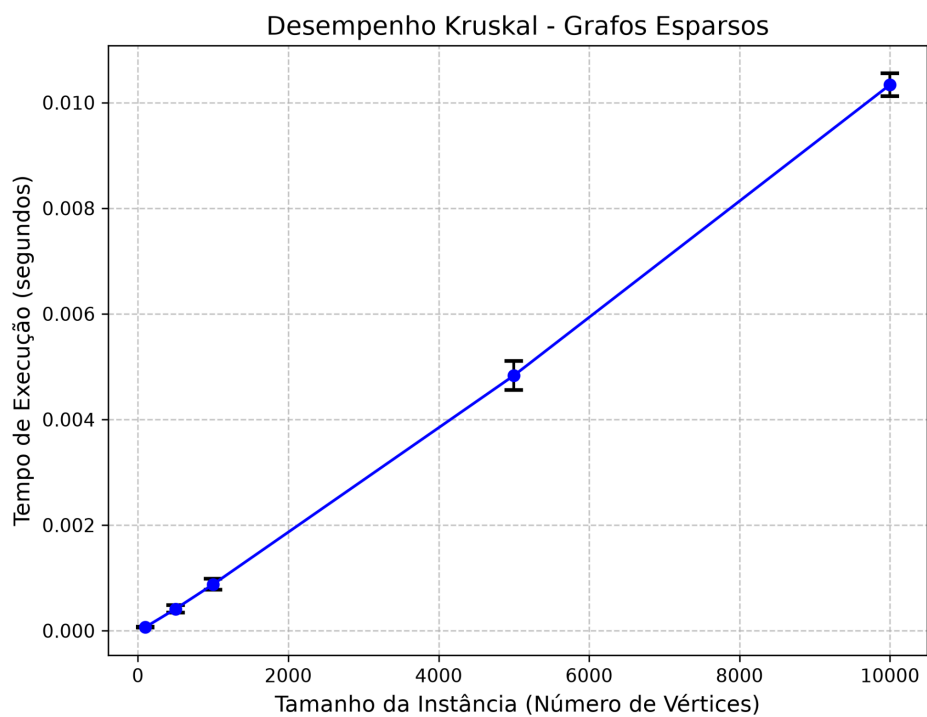


Figura 1

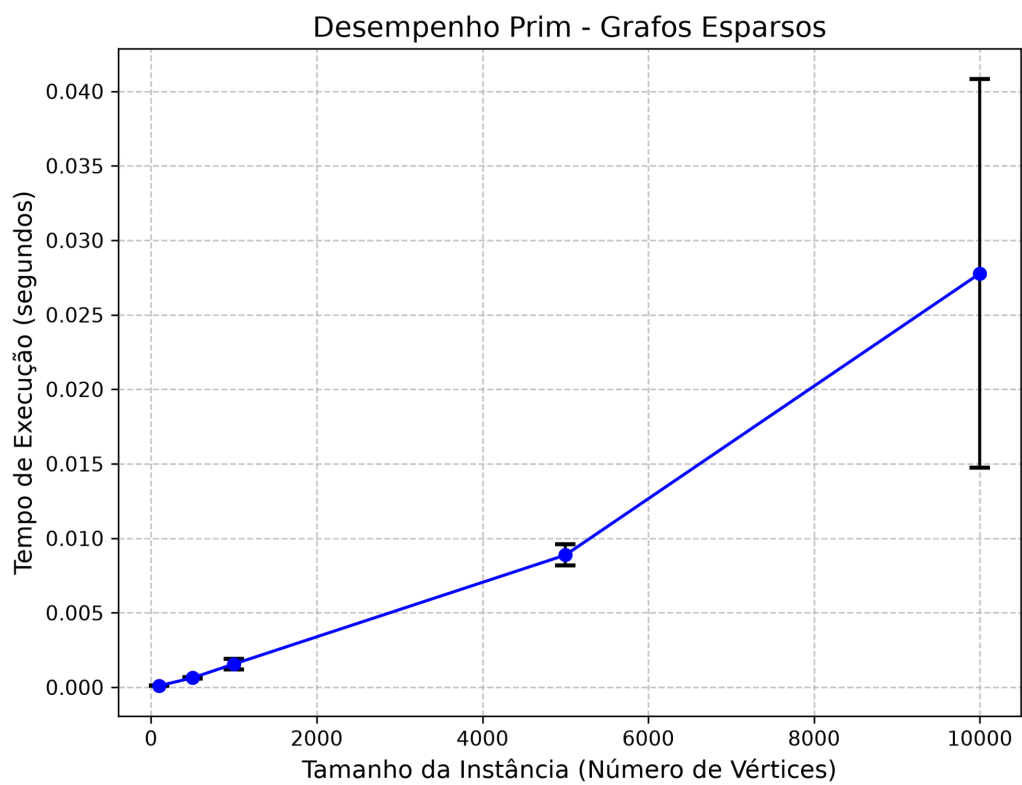


Figura 2

Gráfico de desempenho DENSOS

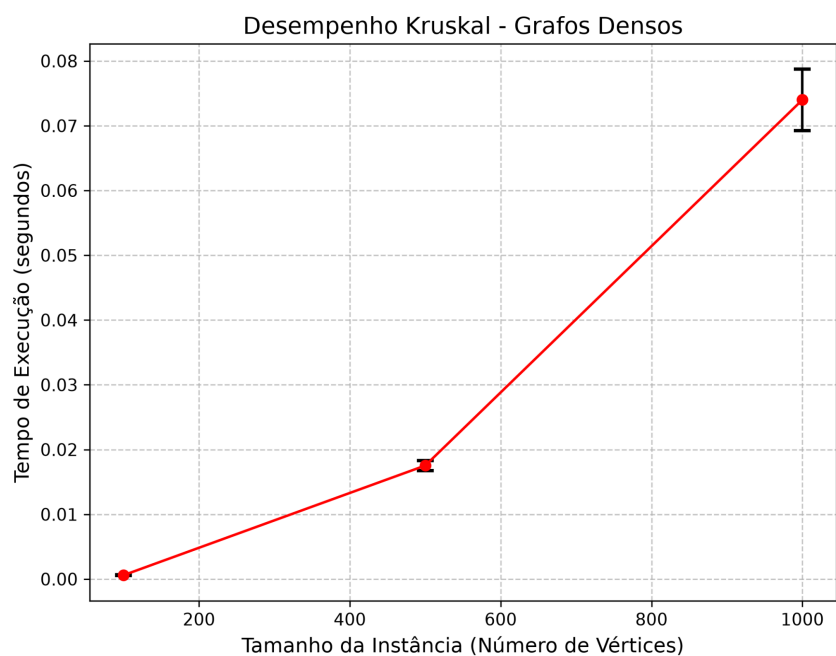


Figura 1

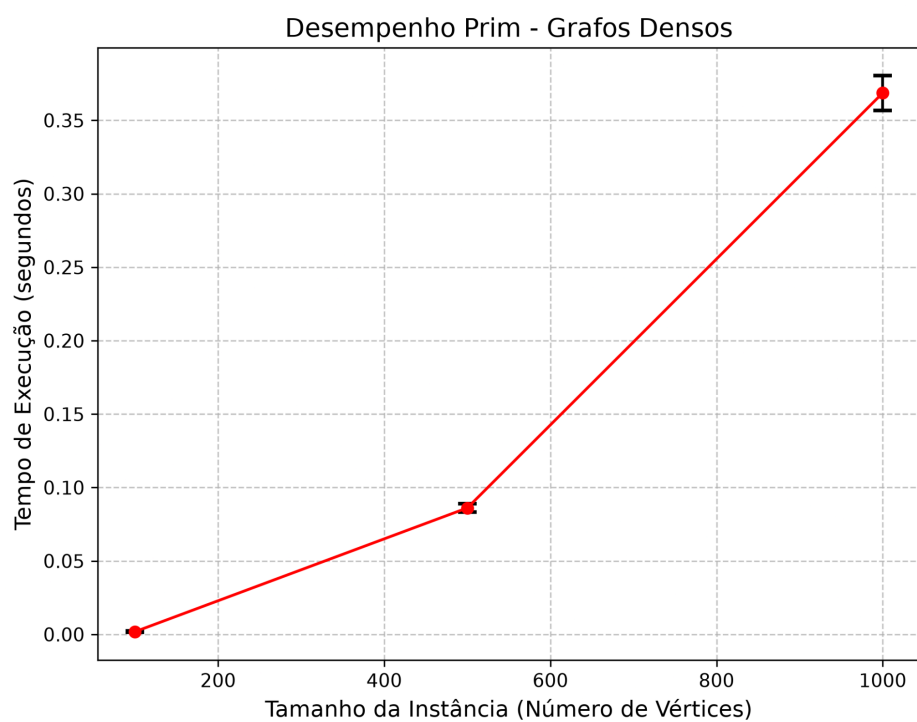


Figura 2

Gráfico de desempenho GEOMÉTRICOS

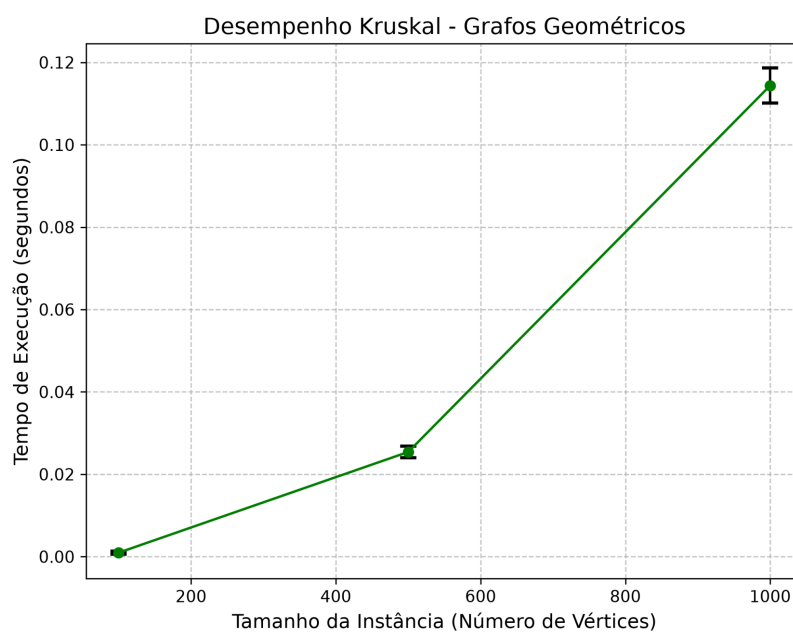


Figura 1

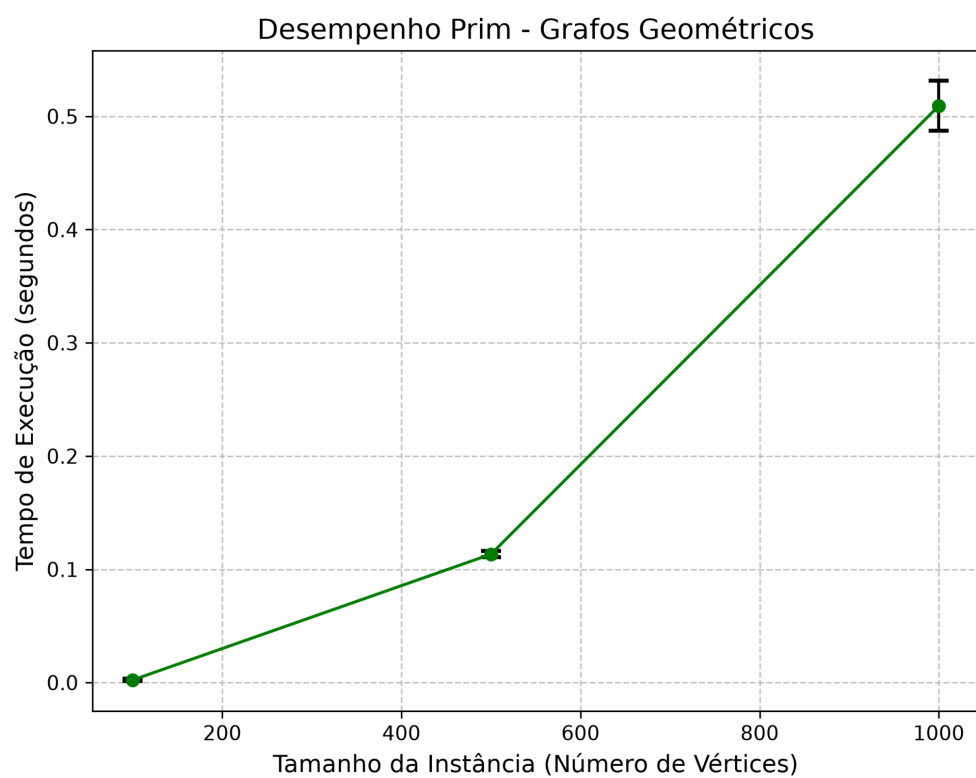


Figura 2

Através da interpretação dos gráficos gerados (onde o eixo horizontal representa o tamanho da instância e o eixo vertical reflete o tempo de execução em segundos), constata-se experimentalmente que:

1. Em Grafos Esparsos:

- a. **Análise do Kruskal (Igor):** O gráfico gerado demonstra uma progressão temporal com feições praticamente lineares, mantendo o tempo de execução na casa de **0,010 segundos** mesmo para a instância de **V=10000**. Esse resultado empírico reflete a exata previsão teórica: como a topologia do grafo esparso impõe que o número de arestas seja reduzido ($|E| \approx |V|$), a etapa de ordenação global que possui complexidade $O(E \log E)$ perde seu peso de processamento, comportando-se na prática como $O(V \log V)$. Combinado a isso, as buscas na estrutura *Union-Find* operam em tempo amortizado quase constante $O(\alpha(V))$, justificando a altíssima velocidade do método neste cenário e as barras de desvio padrão virtualmente nulas.
- b. **Análise do Prim (Ismael):** Olhando para o gráfico do Prim, dá para ver que o tempo aumenta bem mais rápido a partir de 5.000 vértices, chegando a cerca de **0,028 segundos** quando atinge 10.000 vértices. Na teoria pura, Kruskal e Prim deveriam empatar no cenário esparso rodando em $O(V \log V)$, já que o número de arestas é pequeno ($|E| \approx |V|$). Só que, na prática, o Prim demorou um pouco mais que o Kruskal. A explicação técnica para isso está na forma como o Python roda os códigos: o Kruskal usa a função nativa `sorted()` para ordenar as arestas, e essa função roda direto em código C superotimizado por baixo dos panos (usando o algoritmo *Timsort*). Já o Prim precisa rodar laços de repetição (`while` e `for`) em puro Python interpretado para controlar o Min-Heap (`heapq`) a cada vértice visitado. Quando o grafo cresce para 10.000 nós, esse peso do interpretador lidando com tantas tuplas e checagens na memória RAM acaba deixando a curva mais acentuada e aumenta a variação dos tempos, o que explica por que a barra de desvio padrão ficou maior no último ponto (chegando a **0,013 segundos**).

2. Em Grafos Densos:

- a. **Análise do Kruskal (Igor):** O gráfico gerado reflete uma curva de progressão nitidamente superlinear (com feições quadráticas),

escalando de frações de milissegundos no ponto **V=100** para **0,074 segundos** no ponto **V=1000**. Esse comportamento empírico valida a fundamentação teórica: em um grafo denso, onde o número de arestas segue a proporção de $|E| \approx |V|^2$, a instância de 1.000 vértices gera praticamente 500 mil arestas. O custo computacional do Kruskal deixa de ser as operações da estrutura *Union-Find* e passa a ser maciçamente dominado pela ordenação global dessa lista de arestas, cuja complexidade assintótica exibe $O(E \log E)$. A leve dilatação da barra de erro (desvio padrão) no último ponto indica a variância de tempo de alocação de memória do interpretador sob alta carga.

- b. **Análise do Prim (Ismael):** No gráfico de grafos densos, dá para ver que o tempo do Prim subiu bem mais rápido, alcançando a média de **0,368 segundos** no ponto de $V=1000$. Comparando com o Kruskal do Igor, que levou 0,074 segundos, o Prim acabou sendo mais lento nesse cenário de alta densidade. Na teoria de livros-texto, o Prim costuma levar uma vantagem competitiva na densidade porque ele não precisa fazer uma ordenação global de todas as 500 mil arestas de uma vez só; ele apenas varre as conexões ligadas à fronteira ativa usando a Fila de Prioridade. No entanto, o nosso teste prático mostrou como a linguagem influencia o resultado: com o número de arestas crescendo de forma quadrática ($|E| \approx |V|^2$), o algoritmo foi obrigado a realizar centenas de milhares de operações de inserção e remoção no Min-Heap. Como o heapq do Prim roda dentro de laços iterativos em puro Python interpretado, esse custo de processamento linha por linha acabou superando o tempo que o Kruskal levou para ordenar a lista inteira usando o `sorted()`, que roda direto em C compilado por baixo dos panos. Mesmo com essa diferença de tempo, o algoritmo se manteve muito estável e previsível, apresentando um desvio padrão baixo de apenas **0,011 segundos** no ponto máximo.

3. Em Grafos Geométricos:

- a. **Análise do Kruskal (Igor):** Observa-se uma curva geométrica ascendente quase idêntica à dos grafos densos, porém atingindo um teto temporal ligeiramente superior: **0,114 segundos** para a instância de **V=1000**. A semelhança na curvatura ocorre porque o gerador calcula a distância euclidiana entre todos os pares de coordenadas, convertendo a topologia, na prática, em um grafo

completo ($|E| \approx |V|^2$). A justificativa técnica para este cenário ser empiricamente mais lento que o denso comum (**0,114s** contra **0,074s**) reside no tipo de dado: enquanto o grafo denso ordena números inteiros puros, o geométrico obriga o algoritmo a ordenar 500 mil valores de ponto flutuante (*floats*) gerados pelas raízes quadradas, adicionando um *overhead* de processador.

- b. Análise do Prim (Ismael): No cenário dos grafos geométricos, a curva do Prim subiu ainda mais, alcançando a média de 0,509 segundos no ponto de $V=1000$. Comparando com os 0,114 segundos do Kruskal do Igor, o Prim também acabou ficando para trás aqui. Isso acontece porque, como o gerador calcula a distância entre todos os pares de pontos, o grafo geométrico vira um grafo completo na prática, nos jogando de volta no problema de processar 500 mil arestas. Mas a razão principal de o Prim ter sido ainda mais lento aqui do que no cenário denso comum (0,509s contra 0,368s) foi justamente o impacto do cálculo com ponto flutuante (*floats*). Como os pesos são distâncias euclidianas cheias de casas decimais, o interpretador do Python teve um custo muito maior para fazer as comparações lógicas de menor/maior. Toda vez que o algoritmo precisava balancear e reorganizar o Min-Heap via *heapq* em puro Python, comparar esses números quebrados gerou um *overhead* pesado de processamento na nossa estrutura, mostrando que o tipo de dado afeta diretamente o desempenho prático do código. Mesmo com esse esforço extra, o comportamento foi bem constante, fechando com um desvio padrão de apenas 0,022 segundos.

Conclusão

Conclui-se que os objetivos propostos para o trabalho foram integralmente alcançados através da implementação funcional e do benchmark automatizado de ambos os métodos. No entanto, os resultados empíricos trouxeram uma quebra de expectativa muito rica em relação à teoria tradicional de grafos. Embora a literatura aponte o algoritmo de Prim como a escolha ideal para topologias de alta densidade por evitar a ordenação global

de arestas, os testes práticos mostraram que o algoritmo de Kruskal foi superior em todos os cenários avaliados.

Fica demonstrado que a escolha de um algoritmo não deve ser feita de forma abstrata ou baseada apenas na complexidade assintótica de "caixa preta". Na prática computacional, os fatores de implementação da linguagem de programação e as constantes de tempo mudam o jogo: a otimização em código nativo C das funções embutidas do Python (como o *Timsort* no Kruskal) foi capaz de superar o peso do interpretador rodando laços explícitos de manipulação de memória (como o Min-Heap no Prim), mesmo sob cargas massivas de dados. Portanto, o ambiente de execução e a eficiência das bibliotecas internas são tão cruciais para a viabilidade prática da otimização em grafos quanto a própria estrutura teórica do algoritmo.

Uso de Ferramentas de IA

Em cumprimento à Portaria Nº 39/PRPPG/UFC, de 01 de outubro de 2025, e conforme determinado nas diretrizes de integridade acadêmica do trabalho, declara-se a utilização de ferramentas de Inteligência Artificial generativa durante o desenvolvimento deste projeto sob as seguintes condições:

(Igor)

- Ferramentas Utilizadas: Google Gemini.
- Finalidade do Uso: O modelo de IA foi empregado estritamente como assistente de engenharia de software e consultor textual, atuando na depuração de erros lógicos (NameError) durante a escrita do script de automação em lote (main.py); na explicação da sintaxe nativa do pacote estatístico do Python para o cálculo isolado do desvio padrão e na adequação gramatical e estruturação dos tópicos deste relatório técnico conforme a norma culta.
- Prompts Utilizados: "Preciso de ajuda para fazer meu trabalho em equipe, fiquei responsável por implementar o algoritmo de kruskal..."; "Existe comando para desinstalar o ambiente virtual?"; "Poderia me ajudar a criar meu relatório com base em outro relatório do trabalho de grafo que fiz no passado.".
- Garantia de Autoria: Respeitando rigorosamente as vedações institucionais, a inteligência artificial não foi utilizada para gerar conteúdo original de cunho teórico, redigir seções substantivas de interpretação crítica, ou fabricar, alterar, manipular ou "embelezar" dados empíricos e gráficos de desempenho. A concepção lógica dos algoritmos de Kruskal, Prim e da estrutura Union-Find permanece inteiramente sob autoria intelectual dos discentes.

Chat Kruskal: <https://gemini.google.com/share/d6b8e383df77>

(Ismael)

Chat PRIM: <https://share.gemini.google/EVikDlzhVXuF>